Level 7 · about 40 h · Python 3.9+

Project: A countdown generator with a hinted total
Examples run on CPython 3.12.3. **You will be able to:**

- Write a generator that yields one value at a time
- Wrap a def with a decorator
- Read a hint back from __annotations__
- State that hints are not enforced while the program runs

Generators and Type Hints

yield pauses and resumes

A generator produces values one at a time. `list` pulls every yielded value. The reference describes `yield` as the way a generator sends a value to its caller.

```
def countdown(n):
    while n:
        yield n
        n -= 1

print(list(countdown(3)))
```

```
Output (CPython 3.12.3):
[3, 2, 1]
```

A decorator wraps a def

The glossary defines a decorator as a way to return a new object, usually a wrapper, that replaces the original. This wrapper uppercases the returned text. Hints are not involved yet.

```
def shout(fn):
    def wrapped(text):
        return fn(text).upper()
    return wrapped

@shout
```

```
def greet(text):
    return "hi " + text

print(greet("ada"))
```

```
Output (CPython 3.12.3):
HI ADA
```

Hints are readable, not a gate

PEP 484 says type hints are optional and are not required at runtime. CPython stores them on `__annotations__`. It does not reject a call that ignores them. `typing` is the standard-library module for the hint names.

```
def add(a: int, b: int) -> int:
    return a + b

print(add(2, 5))
print(add.__annotations__["return"].__name__)
```

```
Output (CPython 3.12.3):
7
int
```

Check yourself

1. What does `yield` do that `return` does not?
2. What does a decorator return?
3. Does Python stop you when a call ignores a hint?

Sources

- Python reference: Yield expressions
- Python glossary: decorator
- Python docs: typing
- PEP 484 – Type Hints

Code © CypherMonkeyLabs, MIT. Text CC BY-NC-SA 4.0. Educational material, not accredited coursework; no outcome or income promised. Built 2026-10-18.